PER : État de l'art

Romain Chardiny, Yohan Mazzi, Ilias Moutaoukil, Mathias Santos Reis

Tables des matières

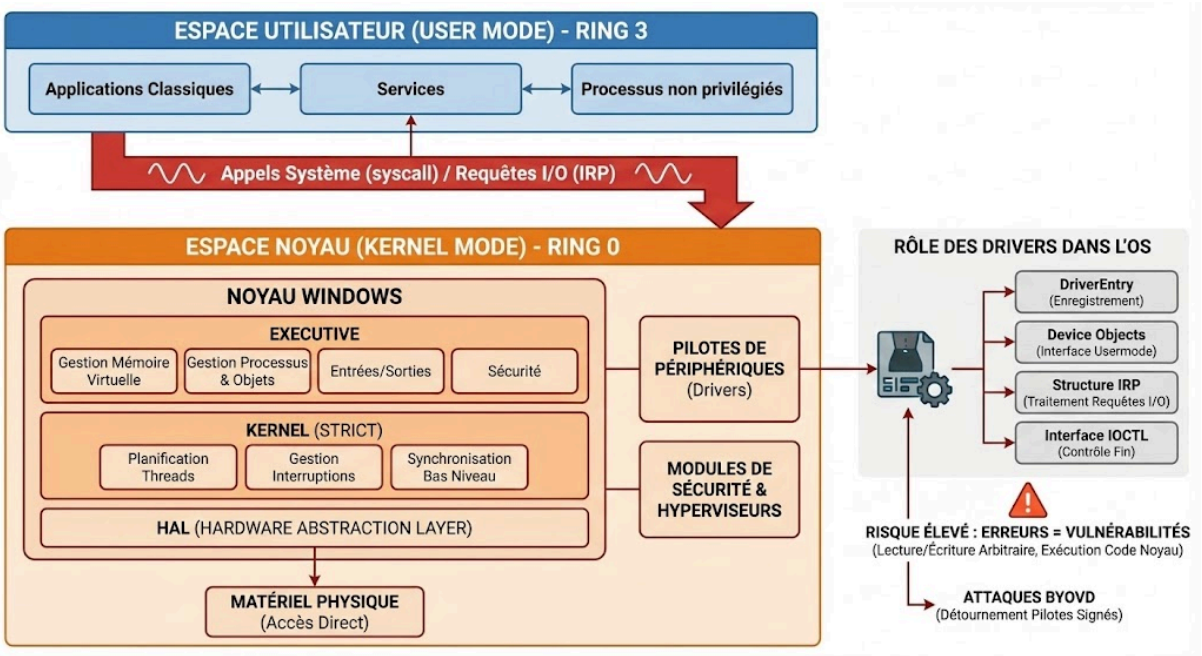
Fonctionnement interne de Windows et intégration des drivers..... 3
Mode utilisateur et mode noyau..... 4
Place et rôle des drivers dans l'OS..... 4
Vulnérabilités de pilotes et risques associés.....5
Causes typiques.....5
Impacts.....5
Le modèle d'attaque BYOVD (Bring Your Own Vulnerable Driver).....6
Fonctionnement typique.....6
Exemples marquants.....6
Contre-mesures Microsoft : blocklist, HVCI, CI.....7
La vulnérabilité CVE-2025-8061 (Lenovo - LnvMSRIO.sys)..... 7
Présentation générale..... 7
Nature technique de la vulnérabilité.....8
Proof of Concept et réplicabilité..... 8
Facteurs d'atténuation et portée pratique.....9
Pertinence pour le projet..... 9
Bibliographie et Références utilisées..... 10
Documentation Microsoft.....10
Bases de données de vulnérabilités.....11
Analyses de pilotes vulnérables..... 11
Blogs techniques de référence..... 11

Notre sujet traite de l'exploitation de l'importation d'un driver vulnérable au sein d'un système (ici un système d'exploitation windows), il faut donc s'informer sur l'état actuel des éléments suivants:

- Le fonctionnement de la stack d'un OS Windows.
- Le fonctionnement d'un driver Windows.
- Les vulnérabilités liées aux drivers Windows et leur exploitation.
- Les contre-mesures existantes.

Fonctionnement interne de Windows et intégration des drivers

Pour comprendre l'impact que peut avoir une attaque de type BYOVD (Bring Your Own Vulnerable Driver) et son fonctionnement, il faut d'abord prendre connaissance de l'environnement dans lequel elle prend place.



Architecture générale de Windows

Le système d'exploitation Windows repose sur une **architecture** dite **hybride**, combinant des concepts issus des micro-noyaux et des noyaux monolithiques. Cette architecture vise à concilier performance, modularité et compatibilité ascendante.

Au plus haut niveau, Windows est structuré autour de deux espaces d'exécution distincts : l'espace utilisateur (*User Mode*) et l'espace noyau (*Kernel Mode*). Cette séparation permet d'isoler les applications et de limiter l'impact des défaillances logicielles sur la stabilité globale du système.

Le noyau Windows est composé de plusieurs sous-composants essentiels :

- le **Kernel** au sens strict, responsable de la planification des threads, de la gestion des interruptions et de la synchronisation bas niveau.
- l'**Executive**, qui fournit des services noyau de plus haut niveau tels que la gestion de la mémoire virtuelle, des processus, des objets, des entrées/sorties et de la sécurité.
- la **HAL (Hardware Abstraction Layer)**, qui isole le noyau des spécificités matérielles.

Les pilotes de périphériques s'intègrent directement dans cette architecture en mode noyau et interagissent étroitement avec **l'Executive** et la **HAL**, ce qui leur confère des privilèges équivalents à ceux du noyau lui-même.

Mode utilisateur et mode noyau

Windows implémente une séparation stricte entre le mode utilisateur (*Ring 3*) et le mode noyau (*Ring 0*), héritée du modèle de protection des processeurs x86/x64.
Les applications classiques, services et processus non privilégiés s'exécutent en mode utilisateur, avec des droits limités et sans accès direct aux ressources matérielles critiques.

Le mode noyau, quant à lui, dispose d'un accès total à la mémoire physique, aux registres processeur sensibles (CRx, MSR), aux périphériques matériels et aux structures internes du système d'exploitation. Les composants exécutés dans ce mode incluent :

- le noyau Windows,
- les pilotes de périphériques,
- certains modules de sécurité et hyperviseurs.

La communication entre ces deux mondes s'effectue via des mécanismes contrôlés tels que les appels système (*syscall/sysenter*) et les requêtes I/O (*IRP*).
Toute violation de cette frontière, volontaire ou accidentelle, peut entraîner une élévation de privilèges ou une compromission complète du système, ce qui explique pourquoi les vulnérabilités en mode noyau sont particulièrement critiques.

Place et rôle des drivers dans l'OS

Les pilotes Windows constituent l'interface logicielle entre le système d'exploitation et le matériel, mais également entre certaines applications et des fonctionnalités noyau spécialisées. Ils sont chargés dynamiquement et exécutés en mode noyau, ce qui leur confère un niveau de privilège maximal.

Un driver Windows typique :

- s'enregistre auprès du noyau via la fonction `DriverEntry`,
- expose éventuellement des *device objects* accessibles depuis l'espace utilisateur,

- traite des requêtes I/O encapsulées dans des structures IRP,
- peut exposer des interfaces IOCTL permettant un contrôle fin depuis l'espace utilisateur.

Cette flexibilité fonctionnelle s'accompagne d'un risque élevé : toute erreur dans la validation des entrées, la gestion mémoire ou le contrôle d'accès peut fournir à un attaquant des primitives dangereuses (lecture/écriture arbitraire, exécution de code noyau).
Dans le cadre des attaques BYOVD, cette confiance implicite accordée aux pilotes signés est exploitée pour détourner leur rôle légitime à des fins malveillantes.

Vulnérabilités de pilotes et risques associés

Connaissant les éléments importants, leur fonction et fonctionnement, on peut se pencher sur leurs potentielles failles de sécurité et les conséquences qui en découlent.

Causes typiques

Les vulnérabilités affectant les pilotes Windows trouvent généralement leur origine dans des erreurs de conception ou d'implémentation, souvent liées à la complexité du développement en mode noyau. Parmi les causes les plus courantes figurent :

- l'absence ou l'insuffisance de validation des paramètres transmis via les IOCTL ;
- l'utilisation non sécurisée de fonctions sensibles telles que `MmMapIoSpace, memcpy`, ou l'accès direct à la mémoire physique ;
- une mauvaise séparation entre buffers utilisateur et buffers noyau ;
- l'exposition de fonctionnalités de debug ou de maintenance non protégées ;
- des contrôles d'accès inexistants ou trop permissifs sur les *device objects*.

Ces failles sont fréquemment introduites dans des pilotes développés par des éditeurs tiers, parfois sans audit de sécurité approfondi, mais bénéficiant néanmoins d'une signature valide reconnue par Windows.

Impacts

Les conséquences d'une vulnérabilité exploitable dans un pilote noyau sont particulièrement sévères. Elles incluent notamment :

- une **élévation de privilèges locale (LPE)** menant à l'obtention des droits SYSTEM ;

- la possibilité de **désactiver ou contourner des mécanismes de sécurité** tels que les EDR, antivirus ou PPL ;

- l'installation de **rootkits noyau** assurant une persistance durable et furtive ;

- l'accès à des primitives avancées de manipulation du noyau (tokens, tables de pages, MSR, structures EPROCESS).

Ces impacts expliquent pourquoi les pilotes vulnérables sont activement recherchés et réutilisés dans des campagnes malveillantes avancées, notamment via le modèle BYOVD.

Le modèle d'attaque BYOVD (Bring Your Own Vulnerable Driver)

Pour pouvoir exploiter ces vulnérabilités il existe donc en particulier l'attaque qui nous intéresse, le modèle BYOVD:

Fonctionnement typique

Une attaque BYOVD suit généralement une chaîne d'exploitation bien définie :

1. L'attaquant identifie un pilote signé et vulnérable, souvent distribué par un éditeur légitime.

2. Le pilote est introduit sur le système cible et chargé, profitant de la confiance accordée par le mécanisme de signature des drivers.

3. L'attaquant communique avec le pilote via un *device object* exposé et envoie des requêtes IOCTL malicieuses.

4. La vulnérabilité fournit une primitive critique (lecture/écriture arbitraire, exécution de code noyau).

5. Le contrôle du noyau est utilisé pour désactiver des protections, élever les privilèges ou assurer une persistance furtive.

Ce modèle est particulièrement efficace car il contourne les politiques de sécurité basées sur la confiance accordée aux composants signés.

Exemples marquants

De nombreux cas documentés illustrent l'utilisation du BYOVD dans des attaques réelles :

- **Slingshot APT** : exploitation d'un driver MikroTik vulnérable pour obtenir un accès noyau persistant.

- **BlackByte ransomware** : utilisation de drivers MSI vulnérables pour neutraliser les EDR.

- **CVE-2021-21551 (Dell)** : primitive d'écriture arbitraire largement exploitée par des malwares.

- **Drivers GIGABYTE** : fonctionnalités non documentées permettant la modification directe du noyau.

Ces exemples démontrent que le BYOVD est devenu une technique standard dans l'arsenal des attaquants modernes.

Contre-mesures Microsoft : blocklist, HVCI, CI

Face à la multiplication des attaques BYOVD, Microsoft a introduit plusieurs mécanismes de défense :

- la **Driver Blocklist**, distribuée via Windows Update, empêche le chargement de pilotes connus comme vulnérables ;

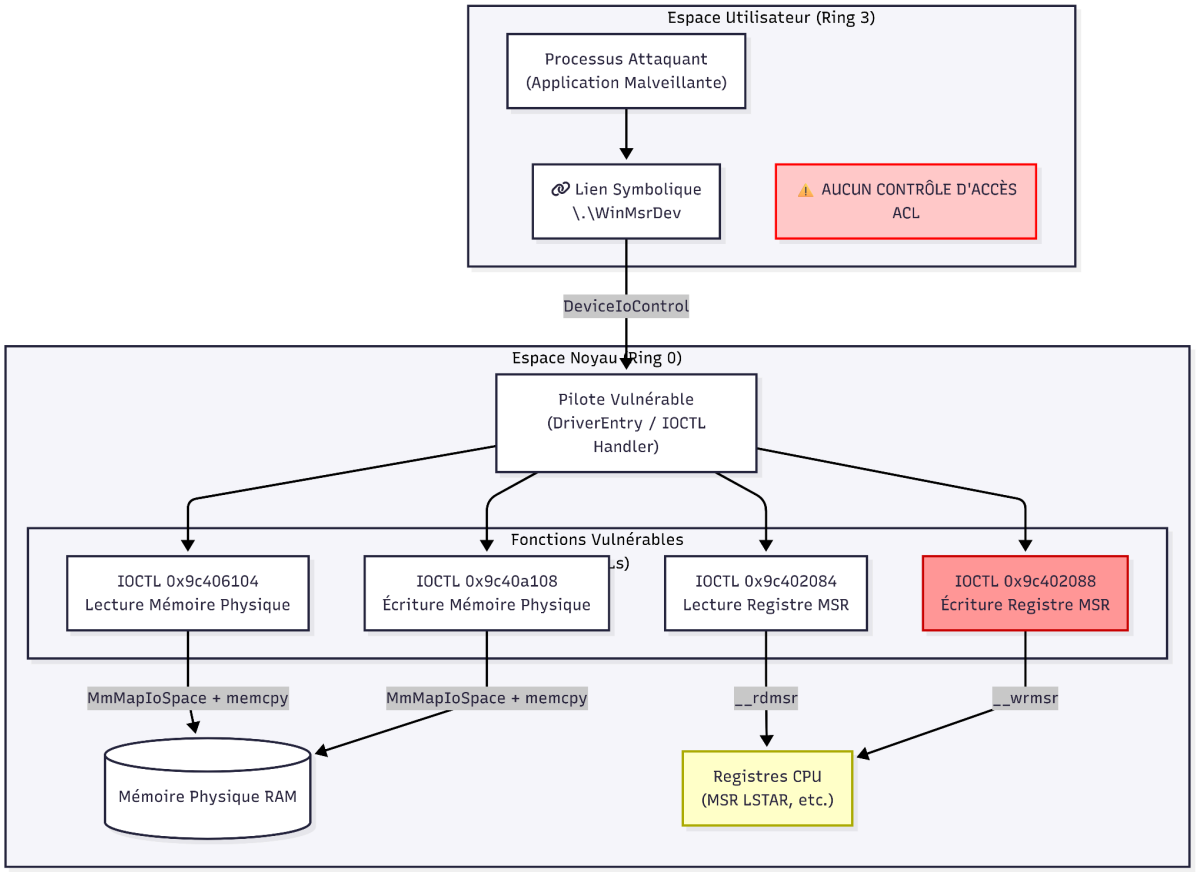
- **HVCI (Hypervisor-Protected Code Integrity)** utilise la virtualisation pour empêcher la modification du code noyau même en cas de compromission partielle ;

- les mécanismes de **Code Integrity (CI)** et *Memory Integrity* renforcent les contrôles sur les composants exécutés en mode noyau.

Cependant, ces protections ne sont pas universellement activées et peuvent être contournées dans certains contextes (systèmes non à jour, versions antérieures de Windows, pilotes récemment divulgués).

La vulnérabilité CVE-2025-8061 (Lenovo - LnvMSRIO.sys)

Cette vulnérabilité constitue un exemple récent et représentatif des dérives possibles liées aux pilotes noyau vulnérables. Son analyse permet d'illustrer concrètement les concepts abordés dans les sections précédentes, en mettant en évidence la chaîne complète d'un scénario BYOVD moderne, depuis l'exposition d'IOCTL non sécurisés jusqu'à l'exécution de code en Ring 0.



Présentation générale

La vulnérabilité **CVE-2025-8061** affecte le pilote noyau **LnvMSRIO.sys**, distribué par Lenovo et intégré à certains packages logiciels du constructeur, notamment le composant *Lenovo Dispatcher* ou *Lenovo Process Management*.
Ce pilote, signé et reconnu comme légitime par Windows, est chargé en mode noyau sur les systèmes concernés.

La faille a été rendue publique en **septembre 2025** et référencée dans la **National Vulnerability Database (NVD)** comme une vulnérabilité de type *insufficient access control*. Elle permet, pour un **utilisateur local authentifié**, d'interagir de manière non restreinte avec des fonctionnalités sensibles du pilote, conduisant potentiellement à une **élévation de privilèges locale** jusqu'au niveau **NT AUTHORITY\SYSTEM**.

Nature technique de la vulnérabilité

Les analyses techniques publiées, notamment par Quarkslab, montrent que la vulnérabilité résulte principalement d'un **manque de contrôles d'accès et de validation des paramètres** dans le traitement des requêtes **DeviceIoControl** exposées par le pilote LnvMSRIO.sys.

Le pilote crée un *device object* accessible depuis l'espace utilisateur et implémente plusieurs **IOCTL** permettant :

- la lecture arbitraire de la mémoire physique ou noyau,
- l'écriture arbitraire dans la mémoire physique ou noyau,
- la lecture et l'écriture de registres sensibles du processeur, notamment des **Model-Specific Registers (MSR)**.

Ces primitives dangereuses sont accessibles sans vérification suffisante du contexte de l'appelant, ce qui permet :

- des **fuites d'adresses noyau**, facilitant le contournement de mécanismes tels que **kASLR** ;
- l'obtention de primitives d'**écriture arbitraire en Ring 0**, ouvrant la voie à une compromission complète du noyau ;
- la manipulation de registres critiques (ex. MSR LSTAR), permettant l'exécution de code arbitraire en contexte noyau.

L'ensemble de ces failles place le pilote dans la catégorie des drivers exploitables dans un scénario BYOVD classique.

Proof of Concept et réplicabilité

À la suite de la divulgation de la vulnérabilité, plusieurs **Poc** ont été rendus publics dans des dépôts de recherche et sur des plateformes collaboratives telles que GitHub.
Ces PoC ciblent notamment des versions vulnérables du pilote, telles que **LnvMSRIO.sys v3.1.0.36**, et démontrent de manière reproductible l'élévation de privilèges locale.

Les chaînes d'exploitation observées exploitent les primitives de lecture et d'écriture noyau afin de :

- détourner le flux d'exécution en Ring 0,
- modifier des structures noyau sensibles (ex. structures EPROCESS),
- voler ou remplacer le **token de sécurité** du processus SYSTEM.

Les travaux publiés par Quarkslab constituent une référence centrale, fournissant une analyse approfondie du fonctionnement interne du pilote et servant de base à la majorité des PoC existants. Ces implémentations confirment la **réplicabilité fiable** de la vulnérabilité dans un environnement de laboratoire.

Facteurs d'atténuation et portée pratique

La vulnérabilité **CVE-2025-8061** n'affecte pas les versions corrigées du composant Lenovo concerné. Les correctifs publiés par Lenovo, notamment dans les versions ultérieures du package *Dispatcher* (version **3.2 et suivantes** selon les advisories), corrigent les failles identifiées.

Par ailleurs, certains mécanismes de sécurité Windows permettent de limiter, voire d'empêcher, l'exploitation :

- l'activation de **Core Isolation / Memory Integrity (HVCI)** bloque certaines formes de modification du noyau ;
- l'intégration éventuelle du pilote dans la **Driver Blocklist** de Microsoft empêche son chargement sur les systèmes à jour.

Néanmoins, sur des systèmes où ces protections ne sont pas activées ou à jour, la vulnérabilité demeure pleinement exploitable, ce qui renforce sa pertinence dans un contexte BYOVD réel.

Pertinence pour le projet

La vulnérabilité CVE-2025-8061 constitue un **cas d'école du modèle BYOVD** : un pilote signé et légitime, des interfaces IOCTL exposant des primitives dangereuses, et une exploitation possible dès lors que certaines protections du système ne sont pas actives.

Son étude s'inscrit pleinement dans les objectifs du projet, en permettant :

- d'analyser une chaîne d'exploitation noyau complète,
- de comprendre les interactions entre drivers vulnérables et mécanismes de défense Windows,
- d'évaluer la visibilité et les réactions des solutions **EDR/AV** face à une exploitation BYOVD réaliste.

Cette vulnérabilité offre ainsi un support pédagogique et technique particulièrement adapté à l'étude expérimentale de l'exploitation de pilotes vulnérables sous Windows.

Synthèse de l'état de l'art

L'analyse de la littérature et des travaux existants met en évidence plusieurs constats majeurs.
Les pilotes Windows représentent une surface d'attaque critique en raison de leur niveau de privilège et de la confiance qui leur est accordée par le système. Le modèle BYOVD exploite précisément cette confiance, en s'appuyant sur des pilotes légitimes mais vulnérables pour contourner les mécanismes de défense modernes.

Malgré les efforts de Microsoft pour renforcer la sécurité du noyau (blocklist, HVCI, CI), les attaques BYOVD demeurent pertinentes, notamment sur des systèmes partiellement protégés ou non à jour. La vulnérabilité CVE-2025-8061 s'inscrit pleinement dans cette problématique et constitue un cas d'étude pertinent pour analyser les mécanismes d'exploitation noyau, les contournements de protections et les limites actuelles des solutions de détection.

Cet état de l'art justifie ainsi pleinement la démarche expérimentale du projet, fondée sur la reproduction contrôlée d'une attaque BYOVD réelle et l'analyse de ses implications techniques et défensives.

Bibliographie et Références utilisées

Documentation Microsoft

- Documentation Windows pour les développeurs de drivers (principale page d'entrée). https://learn.microsoft.com/en-us/windows-hardware/drivers/
- Documentation de l'API pour le Kit de pilotes Windows (WDK) — références détaillées des API. https://learn.microsoft.com/fr-fr/windows-hardware/drivers/ddi/
- Liste de vérification de la sécurité des pilotes — guide de bonnes pratiques pour des drivers sécurisés. https://learn.microsoft.com/fr-fr/windows-hardware/drivers/driversecurity/driver-security-checklist

Bases de données de vulnérabilités

- NIST National Vulnerability Database — CVE-2025-8061 (fiche détaillée de la vulnérabilité étudiée). https://nvd.nist.gov/vuln/detail/CVE-2025-8061
- MITRE CVE List — CVE repository (catalogue général des vulnérabilités). https://learn.microsoft.com/fr-fr/windows-hardware/drivers/driversecurity/threat-modeling-for-drivers
- MITRE CWE — Common Weakness Enumeration (classification des types de failles). https://cwe.mitre.org/

Analyses de pilotes vulnérables

- JVNTA#90371415 — Multiple third-party kernel drivers for Windows vulnerable to improper access control on IOCTL (analyse vulnérabilités IOCTL). https://jvn.jp/en/ta/JVNTA90371415/index.html

- Rapport de Kaspersky sur l'augmentation des attaques utilisant des pilotes vulnérables (contexte BYOVD attaques récents). https://www.kaspersky.fr/about/press-releases/kaspersky-detecte-une-augmentation-

de-23-des-attaques-visant-les-pilotes-windows-vulnerables

- Qilin Ransomware – BYOVD case study (exemple d'exploitation BYOVD dans la nature).
  https://cyberveille.esante.gouv.fr/actualites/qilin-ransomware-la-menace-du-bring-your-own-vulnerable-driver-2025-08-12

- A. Monzani, A. Parata, A. Oliveri, S. Aonzo, D. Balzarotti, et A. Lanzi, "Unveiling BYOVD Threats: Malware's Use and Abuse of Kernel Drivers" dans Proceedings of the Network and Distributed System Security (NDSS) Symposium, San Diego, CA, USA, 2026. doi: 10.14722/ndss.2026.231491.

Blogs techniques de référence

- Quarkslab – Analyse technique de CVE-2025-8061 (article de référence utilisé pour ton étude).
  https://blog.quarkslab.com/exploiting-lenovo-driver-cve-2025-8061.html
- Github - symeonp/Lenovo-CVE-2025-8061 (POC du CVE-2025-8061)
  https://github.com/symeonp/Lenovo-CVE-2025-8061